Python/Pytest Automation Test Sheet

Webinar 6 — execution, triage and evidence

Alexandra Holoborodko

https://github.com/alexandramoroz12/embeded_firmware-autotests_hw6/

Environment

Field	Value
Python	Python 3.14.7
pytest	pytest 9.1.1
pyserial	pyserial 3.5
DEVICE_PORT	rfc2217://localhost:4000
Firmware	BUGGY
Serial	115200

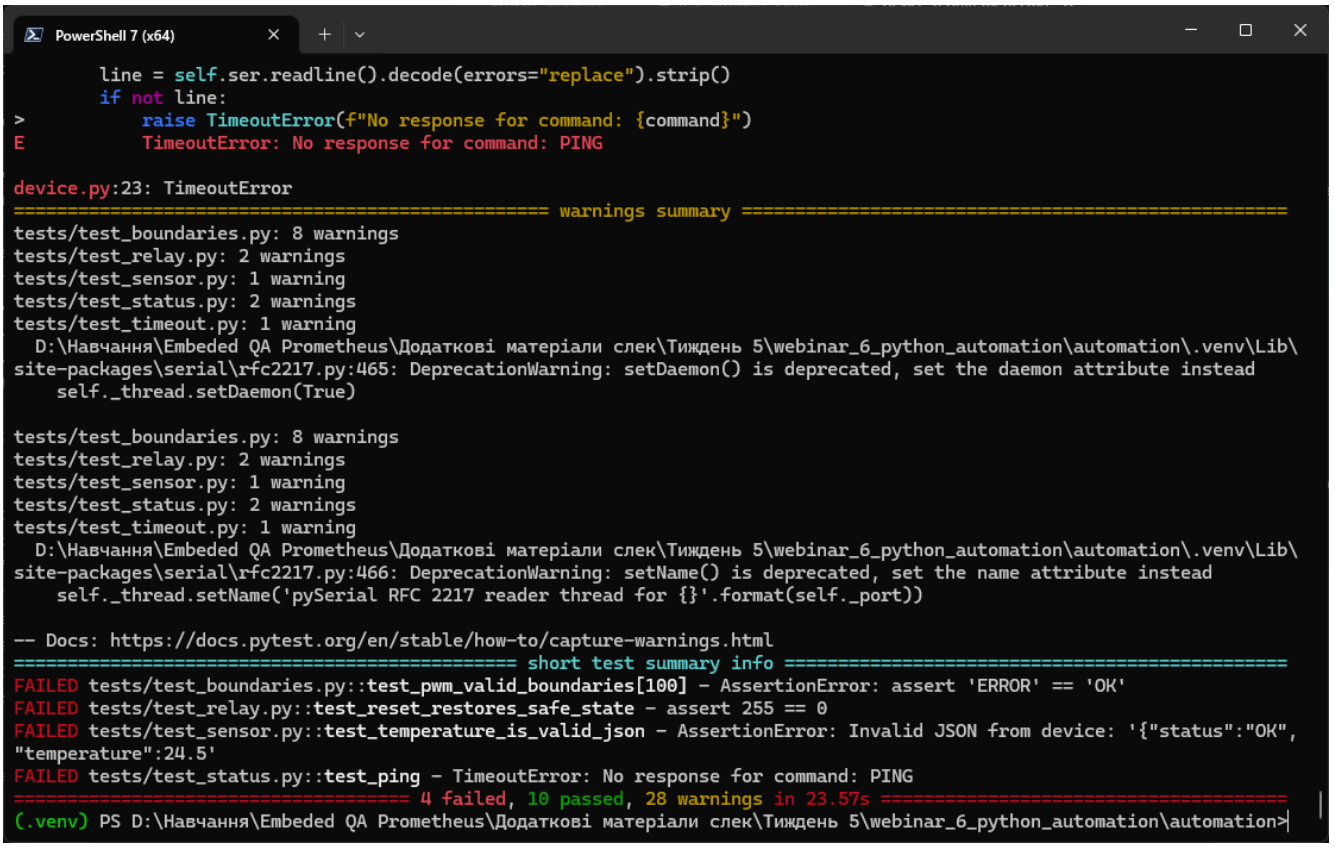
Execution Log

ID	Test / Input	Expected	Actual	Classification	Evidence	P/F
T-01	test_boundaries:test_pwm_valid_boundaries	Tests with valid boundaries are passed	FAILED tests/test_boundaries.py::test_pwm_valid_boundaries[100] - AssertionError: assert 'ERROR' == 'OK'	firmware	Pytest run	F bug_1
T-02	test_boundaries:test_pwm_invalid_boundaries	Tests with invalid boundaries are passed	Tests are passed	-	Pytest run	P
T-03	test_relay:test_led_on_off	Tests are passed	Tests are passed	-	Pytest run	P
T-04	test_relay:test_reset_restores_safe_state	Tests are passed	FAILED tests/test_relay.py::test_reset_restores_safe_state - assert 255 == 0	firmware	Pytest run	F bug_2
T-05	test_sensor: test_temperature_is_valid_json	Tests are passed	FAILED tests/test_sensor.py::test_temperature_is_valid_json - AssertionError: Invalid JSON from device: '{"status":"OK","temperature":24.5'	firmware	Pytest run	F bug_3
T-06	test_status:test_ping	Tests are passed	FAILED tests/test_status.py::test_ping - TimeoutError: No response for command: PING PING returns valid JSON {"status":"OK","command":"PING"}, so requirement 1 is met. However, the response arrives after ~1.3 s (6 of 6 measurements: 1.28-1.34 s), while STATUS responds in ~0.17 s. The test waits for a response for 1.0 s (timeout=1.0 in conftest.py), so it fails with TimeoutError	Communication (client timeout shorter than the device response time; no response-time requirement exists)	With timeout=5.0, test_ping passes. Output of check.py with per-comm and Response times shows that the response does arrive, just late. PING also responds correctly in the Wokwi Serial Monitor.	F bug_4
T-07	test_status:test_status	Tests are passed	Tests are passed	-	Pytest run	P
T-08	test_timeot: test_no_response_becomes_timeout	Tests are passed	Tests are passed	-	Pytest run	P
T-09	test_my_cases:test_pwm_invalid_values	Tests are passed	Tests are passed	-	Pytest run	

T-10	test_my_cases:test_unknown_commands_return_structured_error	Tests are passed	Tests are passed	-	Pytest run	P
T-11	test_my_cases:test_timeout_when_no_response	Tests are passed	Tests are passed	-	Pytest run	P

FAIL Classification

Category	Check
Test	Assert/oracle correct?
Environment	DEVICE_PORT/dependencies/port ownership?
Communication	Timeout/transport/raw bytes?
Requirement	Expected explicitly supported?
Firmware	Repeatable mismatch with requirement?



Bug Reports

Bug_1

Title: “Invalid value” error is returned after entering `SET PWM 100` command

Requirement: `SET PWM N` accepts every integer from 0 through 100 inclusive.

Environment: ESP32 DevKit

Preconditions: Device ACTIVE

Test/Steps:

1. Enter `SET PWM 100` command

2. Check log

Expected: {"status":"OK","pwm":100}

Actual: “Invalid value” error is returned

Raw Serial Evidence: {"status":"ERROR","code":"INVALID_VALUE"}

Pytest Evidence: FAILED tests/test_boundaries.py::test_pwm_valid_boundaries[100] - AssertionError: assert 'ERROR' == 'OK'

Severity: critical

Reproducibility: 5/5



Bug_2

Title: RESET does not restore PWM to 0 and LED to OFF

Requirement: RESET restores safe output state: LED/PWM OFF/0

Environment: ESP32 DevKit

Preconditions: Device ACTIVE

Test/Steps:

1. Enter SET LED ON command
2. Enter RESET command
3. Enter STATUS command
4. Check log

Expected: PWM is 0 and LED is OFF

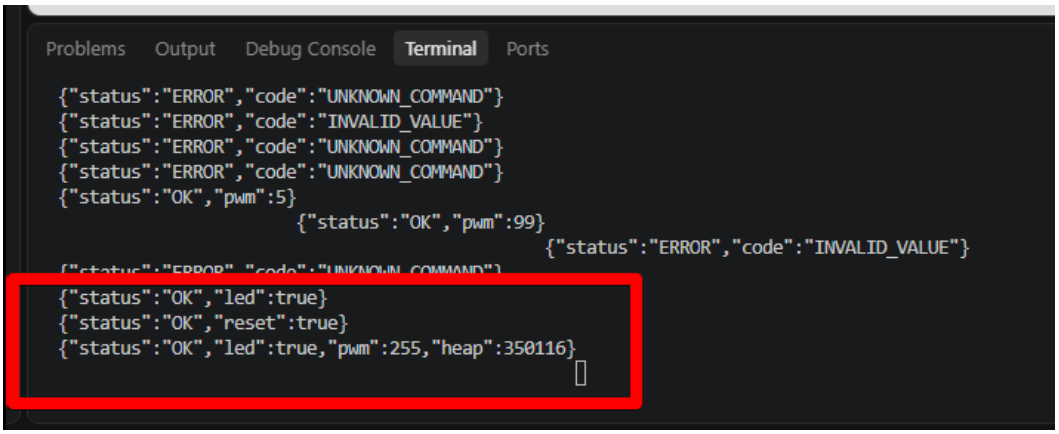
Actual: PWM is 255,LED is ON

Raw Serial Evidence: {"status":"OK","led":true,"pwm":255,"heap":350116}

Pytest Evidence: FAILED tests/test_relay.py::test_reset_restores_safe_state - assert 255 == 0

Severity: critical

Reproducibility: 5/5



Bug_3

Title: GET TEMP returns malformed JSON

Requirement: GET TEMP returns valid JSON with either a valid temperature or an explicit sensor error

Environment: ESP32 DevKit

Preconditions: Device ACTIVE

Test/Steps:

1. Enter GET TEMP command

2. Check log

Expected: The device returns valid JSON containing either a valid temperature with status OK or an explicit sensor error with status ERROR.

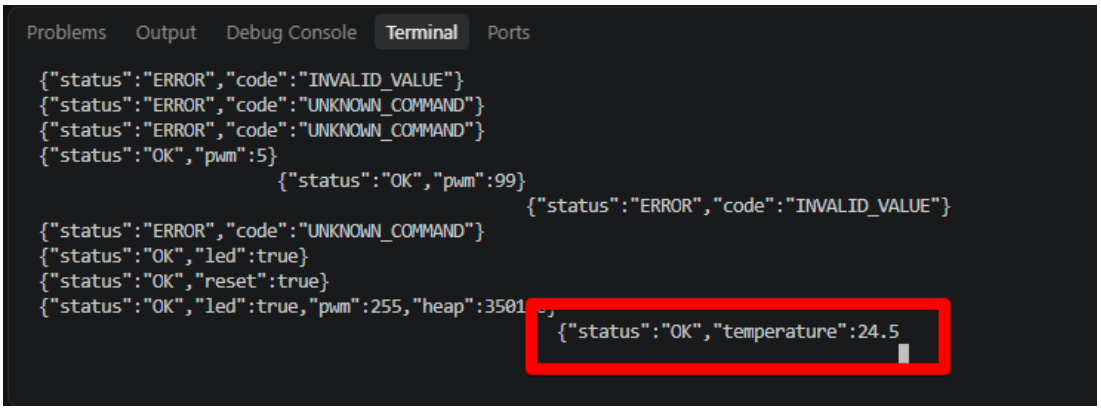
Actual: device returns malformed JSON: {"status":"OK","temperature":24.5 — the closing } is missing

Raw Serial Evidence: {"status":"OK","temperature":24.5

Pytest Evidence: FAILED tests/test_sensor.py::test_temperature_is_valid_json - AssertionError: Invalid JSON from device: '{"status":"OK","temperature":24.5'

Severity: Low/Medium

Reproducibility: 5/5



Bug_4

Title: PING response is delayed to ~1.3 s (vs ~0.17 s for other commands), causing TimeoutError with a 1.0 s client timeout

Requirement: Automation must never wait forever: communication uses a finite timeout.

Environment:

- Firmware running in Wokwi simulation (ESP32), serial access via rfc2217://localhost:4000

- Python 3.14.7, pytest 9.1.1, pyserial

- Test harness: Device class, timeout=1.0 (set in conftest.py)

Preconditions: Device ACTIVE

Test/Steps:

1. Start the Wokwi simulation and connect over RFC2217 (115200 baud).

2. Send PING\n.

3. Measure the time until the response line arrives.

4. For comparison, send STATUS\n and measure the time.

5. Or run python -m pytest tests/test_status.py::test_ping -v with timeout=1.0.

Expected:

PING responds with {"status":"OK","command":"PING"} in a time comparable to other commands (~0.2 s), i.e. within the 1.0 s client timeout.

Actual:

The response is correct, but arrives after 1.28-1.34 s (6 of 6 measurements). With a 1.0 s timeout, the client raises TimeoutError: No response for command: PING.

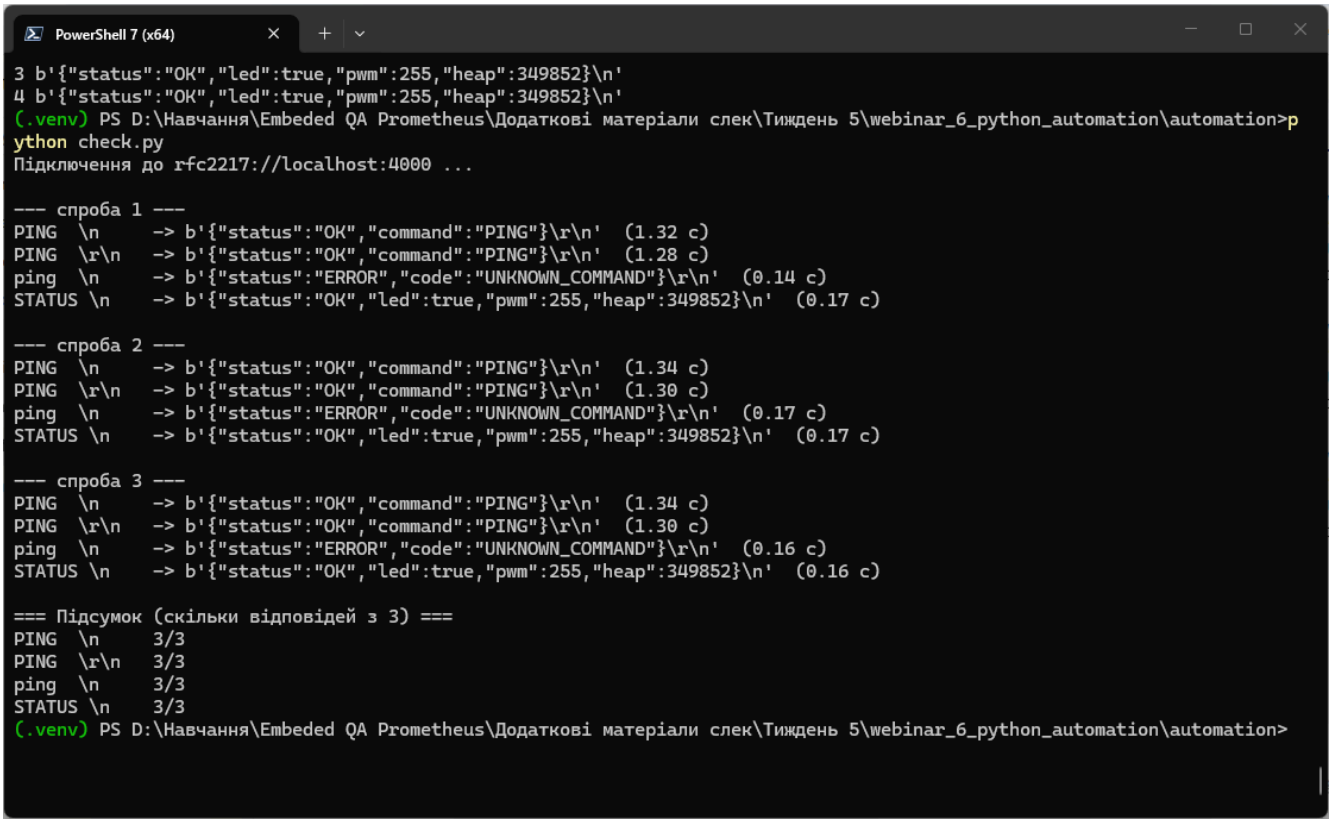
NOTE: PING responds with {"status":"OK","command":"PING"} if enter command manually in Wokwi Terminal

Raw Serial Evidence: -

Pytest Evidence: FAILED tests/test_status.py::test_ping - TimeoutError: No response for command: PING

Severity: Low/Medium. The functional response is correct, but the delay breaks clients using a ~1 s timeout.

Reproducibility: 5/5



After adding my tests

